

Kiến trúc NLU trên nền Cube Core (Semantic / Query Engine)

Text-to-SQL Agent cho nền tảng dữ liệu Smart City. LLM cụ thể **chưa chốt** trong tài liệu này — thiết kế ở mức interface chung, chọn provider là quyết định sau.

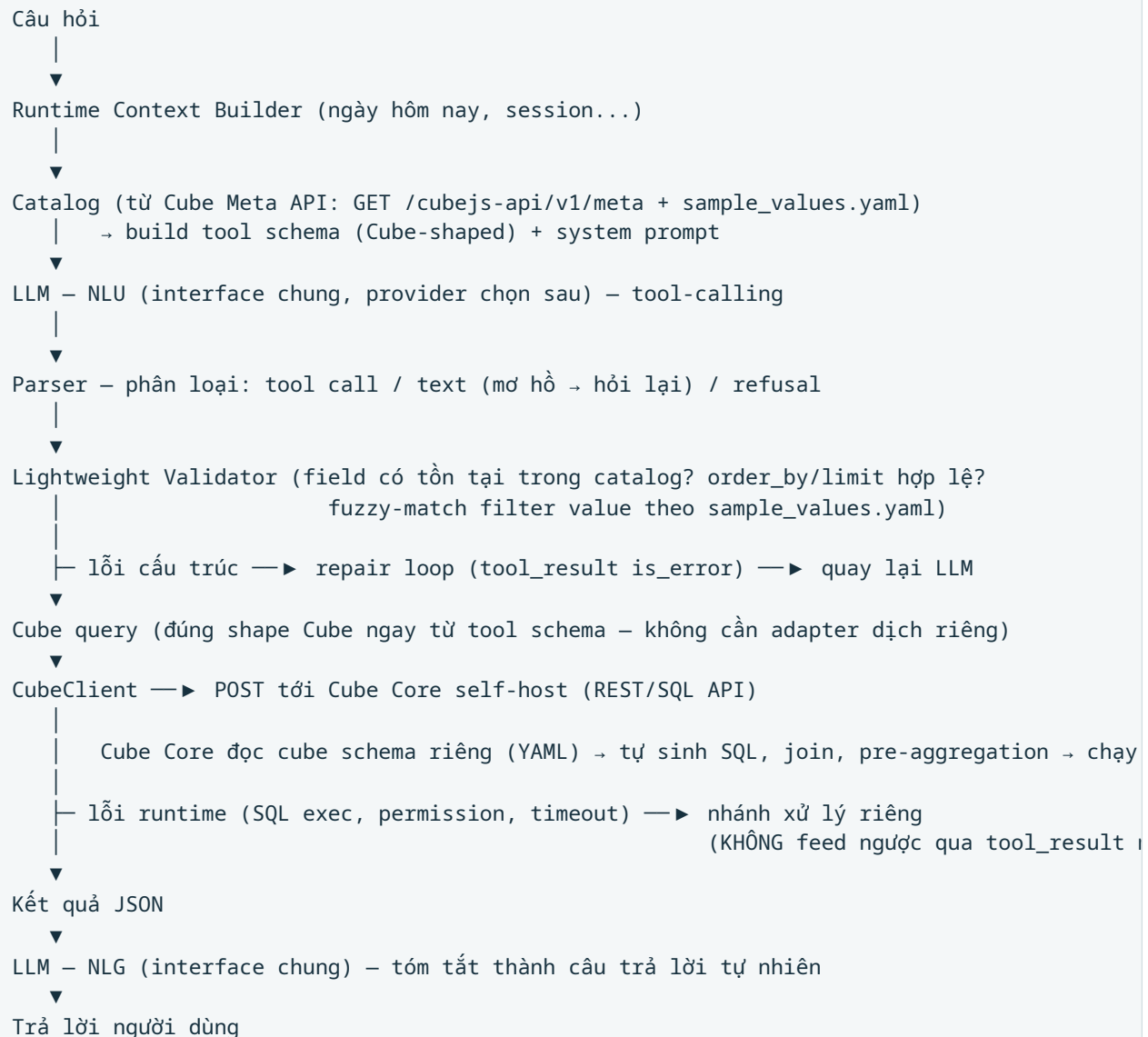
1. Mục tiêu kiến trúc

Hệ thống nhận câu hỏi tiếng Việt/tự nhiên về dữ liệu Smart City (điện, giao thông, chất lượng không khí, giao thông công cộng...) và trả lời bằng cách:

1. LLM diễn giải câu hỏi thành một structured query đúng shape của Cube Core (function-calling), không tự viết SQL.
2. Structured query được validate ở mức nhẹ trước khi gửi đi.
3. Cube Core (self-host) nhận query, tự sinh SQL, xử lý join/pre-aggregation, chạy trên data warehouse.
4. LLM tóm tắt kết quả trả về thành câu trả lời tự nhiên.

Cube Core được chọn làm semantic + query engine vì self-host miễn phí (Apache-2.0/MIT, không phải qua Cube Cloud) và đã hỗ trợ sẵn join theo quan hệ khai báo, tính measures/dimensions, pre-aggregation, cache, sinh SQL — không cần tự viết một SQL compiler.

2. Kiến trúc tổng quan



Đi qua từng bước với một ví dụ cụ thể

Câu hỏi: "Tổng tiêu thụ điện của quận 1 trong tháng trước là bao nhiêu?"

Nửa đầu — biến câu hỏi thành query, không đụng SQL:

- **Runtime Context Builder** — tách phần "biến động" (ngày hôm nay, session) khỏi phần "ổn định" (catalog, quy tắc). Lý do tách: nếu nhét ngày hôm nay vào chung với system prompt, prefix của prompt đổi mỗi ngày → cache bị vô hiệu mỗi lần request. Thuần là vấn đề tối ưu chi phí/độ trễ, không ảnh hưởng logic.
- **Catalog** — trước khi gọi LLM, hệ thống hỏi Cube Meta API: "hiện có những measure/dimension nào?" Kết quả (vd `energy.total_consumption`, `energy.district_name` ...) dùng để dựng ra hai thứ: **tool schema** (định nghĩa function

`query_metrics` với tham số `measures` / `dimensions` / `filters` / `timeDimensions`, mỗi tham số có `enum` giới hạn đúng bằng danh sách lấy được từ Cube — LLM không thể tự bịa tên field không tồn tại), và **system prompt** (mô tả bằng markdown ý nghĩa từng measure/dimension để LLM hiểu ngữ nghĩa, không chỉ tên).

- **LLM — NLU** — nhận câu hỏi + tool schema, gọi function `query_metrics` với input kiểu:

```
{
  "measures": ["energy.total_consumption"],
  "filters": [{"member": "energy.district_name", "operator": "equals", "values": ["Quận 1"]}],
  "timeDimensions": [{"dimension": "energy.recorded_at", "dateRange": "last month"}]
}
```

Input này đã đúng luôn shape mà Cube hiểu — không cần dịch qua một shape trung gian riêng.

- **Parser** — đọc response của LLM, rẽ 3 nhánh: có gọi tool → đi tiếp xuống validator; chỉ trả lời bằng text (không gọi tool) → LLM đang "từ chối đoán bừa" vì câu hỏi mơ hồ (vd "cho tôi xem số liệu quận 1" — không biết measure nào) → trả thẳng text đó về cho người dùng làm câu hỏi làm rõ, dừng pipeline; refusal (an toàn) → dừng, báo lỗi khác.
- **Lightweight Validator** — chỉ kiểm tra những gì `enum` trong tool schema không bắt được: `order_by` có trở đúng vào field đã chọn không, `limit` có vượt guardrail không, và fuzzy-match giá trị filter (LLM viết "Quận 1", DB có thể lưu `district_1` — validator tra `sample_values.yaml` để quy đổi). Nếu sai cấu trúc, lỗi được gói thành `tool_result` có `is_error: true`, gửi ngược lại LLM — NLU để nó tự sửa (**repair loop**, giới hạn số lần thử). Nếu hợp lệ → đi tiếp.

Nửa sau — Cube thực thi, LLM tóm tắt:

- **Cube query** — kết quả sau validate chính là JSON gửi thẳng cho Cube, không qua bước "biên dịch" nào nữa.
- **CubeClient** → **Cube Core** — gửi HTTP POST tới Cube Core tự host (vd `http://localhost:4000/cubejs-api/v1/load`). Cube Core đọc cube schema YAML của riêng nó (định nghĩa sẵn measures, dimensions, và **joins** giữa các bảng), tự sinh câu SQL đúng (join `energy_consumption` với `district`, filter, aggregate SUM), kiểm tra cache pre-aggregation trước khi chạy thật trên data warehouse.
- **Hai loại lỗi khác nhau:** lỗi *validation* (ở bước trên) LLM sửa được, vì đó là lỗi "chọn sai field/tham số". Lỗi *runtime* từ Cube (SQL timeout, DB down, không có quyền) LLM không sửa được bằng cách gọi lại tool khác, nên đi theo nhánh xử lý riêng (thường là báo lỗi hệ thống cho người dùng, không phải "gọi lại LLM để sửa query").
- **Kết quả JSON → LLM — NLG** — một lượt gọi LLM **riêng biệt** (có thể dùng model nhẹ hơn vì bài toán dễ hơn: tóm tắt số liệu, không cần suy luận phức tạp), nhận JSON kết quả + câu

hỏi gốc, sinh câu trả lời tự nhiên: "Tổng tiêu thụ điện của Quận 1 trong tháng trước là 45.230 kWh."

Điểm cốt lõi cần nhớ: toàn bộ nửa đầu **không có SQL nào được LLM viết ra** — LLM chỉ điền tham số có `enum` giới hạn sẵn. SQL thật sự chỉ được sinh ra ở nửa sau, và việc đó do Cube Core làm, không phải code tự viết. Hai lượt gọi LLM (NLU và NLG) là hai lượt độc lập, có thể dùng model khác nhau tùy độ khó bài toán từng bước.

Khi catalog lớn: thêm bước retrieval trước khi build tool schema

Với catalog nhỏ (vài chục measure/dimension như hiện tại), toàn bộ catalog lấy từ Cube Meta API được đưa thẳng vào tool schema mỗi request, như mô tả ở trên. Khi catalog lớn (hàng trăm/ngày nghìn field), đưa thẳng toàn bộ vào mỗi request làm giảm độ chính xác của LLM (dễ chọn nhầm field giữa một `enum` quá dài) và tốn token mỗi lượt gọi. Lúc đó bước Catalog cần chèn thêm một bước lọc trước khi build tool schema:



Catalog vẫn fetch và cache **toàn bộ** một lần từ Cube Meta API như cũ — retrieval chỉ thay đổi cách catalog đó được đưa vào *tool schema* mỗi request (toàn bộ vs. tập con liên quan). Nếu retrieval không tìm được ứng viên đủ tin cậy cho câu hỏi, hệ thống rơi về đúng cơ chế "không chắc → hỏi lại" đã có sẵn ở bước Parser, thay vì ép LLM chọn trong một danh sách quá lớn hoặc quá hẹp sai hướng.

Với quy mô hiện tại của project (4 cube, ~14 metric), bước retrieval này **chưa cần thiết** — Catalog vẫn build trực tiếp từ toàn bộ Cube Meta API. Đây là điểm mở rộng dành cho giai đoạn catalog phát triển lớn hơn nhiều so với hiện tại.

3. Nguyên tắc "một nguồn"

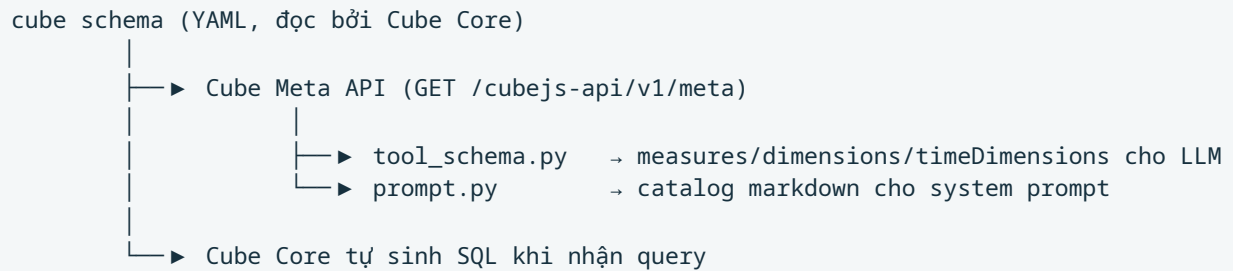
Vấn đề cần giải quyết

Có ba thứ trong hệ thống cần "biết" measures/dimensions nào tồn tại: (1) tool schema gửi cho LLM, (2) system prompt mô tả ngữ nghĩa cho LLM, (3) chính Cube Core khi thực thi query. Nếu

ba chỗ này lấy thông tin từ ba nguồn khác nhau (vd ai đó viết tay tool schema, viết tay prompt, và viết riêng cube schema), chúng **sẽ lệch nhau theo thời gian** — thêm một measure mới vào cube schema nhưng quên cập nhật tool schema, LLM sẽ không bao giờ dùng được measure đó dù dữ liệu đã sẵn sàng.

Cách giải quyết: một nguồn, đọc lại qua API

Cube schema (YAML, do Cube Core đọc và tự validate) là nguồn duy nhất cho measures, dimensions và relationships.



Cube schema chỉ được viết **một lần**, ở một nơi (`cube/model/cubes/*.yaml`). Cube Core đọc nó để biết cách sinh SQL — đây là mục đích chính, không thể thiếu. Nhưng Cube Core còn tự expose một API phụ (`/cubejs-api/v1/meta`) trả về **chính schema đó dưới dạng JSON có thể máy đọc được**. Hệ thống NLU gọi API này lúc khởi động, và dùng kết quả để tự sinh cả tool schema lẫn system prompt.

Hệ quả: thêm một measure mới = sửa một file YAML duy nhất (`cube/model/cubes/*.yaml`). Không có bước "nhớ cập nhật thêm chỗ khác" — vì không có chỗ khác nào viết tay nữa. Tool schema và prompt tự động phản ánh đúng cái Cube Core thực sự có, vì chúng lấy dữ liệu trực tiếp từ Cube, không phải từ một bản sao chép tay.

Vai trò riêng của `sample_values.yaml`

Đây là **ngoại lệ có chủ ý** với nguyên tắc "một nguồn" — nó là nguồn thứ hai, nhưng phạm vi rất hẹp và lý do tồn tại rõ ràng: Cube Meta API trả về *tên* các dimension (vd `energy.district_name`), nhưng không trả về *giá trị thật sự có trong DB* của dimension đó (Cube không có khái niệm "liệt kê giá trị mẫu" tích hợp sẵn). Trong khi đó, validator cần biết "Quận 1" (cách người dùng/LLM gõ) tương ứng với giá trị thật `district_1` trong cột `district_name` để fuzzy-match đúng.

Vì thông tin này Cube không cung cấp, nó phải sống ở một file riêng, nhỏ, chỉ chứa mapping hiển thị↔giá trị thật — không phải toàn bộ semantic model như file YAML tự viết trước đây, chỉ đúng phần Cube thiếu.